

5_FUNÇÕES

October 5, 2026

1 Funções

Funções são blocos de código que realizam uma tarefa específica e podem ser reutilizados sempre que necessário. Isso é essencial para organizar e modularizar o código.

1.0.1 Sintaxe Básica de uma Função

```
[3]: def minha_funcao():  
      pass
```

Um erro bastante comum entre os alunos que estão começando a trabalhar com funções é definir a função, mas esquecer de chamá-la. Em Python, apenas declarar uma função — usando `def` e escrevendo o seu conteúdo — não faz com que o código dentro dela seja executado. A definição apenas informa ao interpretador que aquela função existe e pode ser usada mais tarde. Para que o código realmente rode, é necessário chamar a função pelo seu nome e os parâmetros se houver.

Um ponto importante é a ordem com que as funções são declaradas. Isso porque o interpretador lê o programa de cima para baixo, portanto uma função precisa estar definida antes de ser executada. Isso significa que é possível declarar as funções em qualquer ordem, desde que a chamada a cada uma delas aconteça depois da sua definição. Caso contrário, o Python exibirá um erro informando que o nome da função não foi reconhecido, pois ainda não havia sido carregado na memória no momento da execução.

Neste primeiro momento, o objetivo é focar no entendimento do conceito de função — como ela é definida, chamada e organizada dentro de um programa. Por isso, é mais simples e didaticamente defirmos a função principal (`main`) e em seguida chamarmos ela. Por isso, a partir de agora, todo o código principal será colocado dentro da função `main`.

Mais adiante, quando estivermos confortáveis com o conceito de funções e com a estrutura geral de um programa em Python, introduziremos o `if __name__ == "__main__":`, uma vez que é o jeito mais correto e profissional pois evita que o código principal seja executado automaticamente quando o arquivo for importado como módulo em outro programa;

1.0.2 Funções sem Argumentos

É uma função que não recebe nada como entrada.

```
[2]: #Definição da função  
def mostrar_menu():  
    print("Seja bem vindo!")
```

```

    print("Escolha o tipo de operação que deseja realizar:")
    print(" 1 - Soma")
    print(" 2 - Subtracao")

#Chamando a função a partir da função principal
def main():
    mostrar_menu()

main()

```

Seja bem vindo!

Escolha o tipo de operação que deseja realizar:

1 - Soma

2 - Subtracao

1.0.3 Funções com Argumentos

Lembre que a variável declarada na assinatura da função é uma outra variável, que recebe uma cópia da variável contida no função principal e não a mesma. Perceba no código abaixo que a função **mostrar_menu()** recebe como entrada um nome.

```

[2]: def mostrar_menu(nome):
    print(f"Seja bem vindo, {nome}!")
    print("Escolha o tipo de operação que deseja realizar:")
    print(" 1 - Soma")
    print(" 2 - Subtracao")

def main():
    usuario = input("Digite o seu nome:")
    mostrar_menu(usuario)

main()

```

Digite o seu nome: João

Seja bem vindo, João!

Escolha o tipo de operação que deseja realizar:

1 - Soma

2 - Subtracao

1.0.4 Funções com Retorno de Valor

Uma função também pode retornar algo, para isso usa-se o comando **return**. A função **obter_opcao** retorna a opção escolhida e o resultado é atribuído a uma variável na função **main**.

```

[4]: def exibir_menu(nome):
    print(f"Seja bem vindo, {nome}!")
    print("Escolha o tipo de operação que deseja realizar:")
    print(" 1 - Soma")
    print(" 2 - Subtracao")

```

```
def obter_opcao():
    opcao = int(input("Digite a sua opção:"))
    return opcao

def main():
    usuario = input("Digite o seu nome:")
    exibir_menu(usuario)
    opcao = obter_opcao() #atribui o retorno da função a variável opcao.
    print(opcao)

main()
```

Digite o seu nome: Diego

Seja bem vindo, Diego!

Escolha o tipo de operação que deseja realizar:

1 - Soma

2 - Subtracao

Digite a sua opção: 2

2

1.0.5 Funções com Argumento (Listas, Dicionários..)

Em Python, quando você passa uma estrutura de dados (como listas, dicionários ou objetos) para uma função, a referência ao objeto é passada, e não uma cópia. Isso significa que as alterações feitas na estrutura dentro da função afetam o objeto original, desde que a alteração seja no conteúdo interno do objeto (e não na própria variável que armazena a referência).

```
[5]: def modificar_lista(lista):
      lista.append(4)

      def main():
          minha_lista = [1, 2, 3]
          modificar_lista(minha_lista)

          print(minha_lista) # [1, 2, 3, 4]

      main()
```

[1, 2, 3, 4]

```
[6]: def redefinir_lista(lista):
      lista = [99] # Nova lista criada localmente.

      def main():
          minha_lista = [1, 2, 3]
          redefinir_lista(minha_lista)
```

```
print(minha_lista) # [1, 2, 3]

main()
```

[1, 2, 3]

```
[7]: def adicionar_chave(dicionario, chave, valor):
      dicionario[chave] = valor

      def main():
          meu_dict = {"a": 1, "b": 2}
          adicionar_chave(meu_dict, "c", 3)

          print(meu_dict) # {'a': 1, 'b': 2, 'c': 3}

      main()
```

{'a': 1, 'b': 2, 'c': 3}

```
[8]: def substituir_dicionario(dicionario):
      dicionario = {"novo": 42} # Isso cria um novo dicionário apenas localmente.

      def main():
          meu_dict = {"a": 1, "b": 2}
          substituir_dicionario(meu_dict)

          print(meu_dict) # {'a': 1, 'b': 2}

      main()
```

{'a': 1, 'b': 2}

1.0.6 Função Principal (Main)

Embora Python não exija explicitamente uma função main, o padrão recomendado é usar uma função principal e chamá-la dentro de um bloco if `if __name__ == "__main__":`. Isso também ajuda a diferenciar entre o código que deve ser executado diretamente e o que deve ser importado em outros módulos. O código dentro de if `name == "main"`: só roda quando o arquivo é executado diretamente, não ao ser importado. Isso evita execução indesejada.

```
[16]: def imprimir_saudacao(nome):
      print(f"Olá, {nome}!")

      def main():
          nome = input("Digite seu nome: ")
          imprimir_saudacao(nome)

      if __name__ == "__main__":
```

```
main()
```

Digite seu nome: Diego

Olá, Diego!